

Programming, Data-Analysis, and Machine-Learning Projects

Kundai Farai Sachikonye — github.com/fullscreen-triangle

An overview of representative projects, grouped by the three data modalities of xMDT-HPB — clinical/language, imaging, and laboratory data — followed by the ML engineering and reproducibility work underneath them. All projects are open-source; most are deployed with a live demo. Where a number is given it is validated against a named reference (IEDB, NIST, 1000 Genomes/gnomAD/UK Biobank).

Explainable / auditable diagnostic AI

syndrome — syndrome-seven.vercel.app

Computational-oncology toolkit: neoantigen identification, MHC binding prediction (AUC 0.81, validated on 2,847 IEDB peptides), and immune-escape scoring. The method uses three-channel discrete Fourier transforms on amino-acid physicochemical properties with *zero* free parameters trained on patient data — every prediction is reproducible from the algorithm alone, independent of any training cohort. Developed with KRAS-mutant pancreatic ductal adenocarcinoma (PDAC) immune evasion as a worked clinical case. Runs in-browser (WebGPU), 50 MB, no cluster required. *Relevance*: a concrete instance of *structural* explainability (auditable by construction) as distinct from post-hoc explanation (SHAP/LIME).

crown-prince — github.com/fullscreen-triangle/crown-prince

Pharmacological AI: PK/PD/ADME trajectories and binding affinities derived from molecular geometry, validated 39/39 against NIST with zero empirical parameters. Interpretable-by-construction: predictions are geometric operations, not statistical associations from potentially biased training data.

Imaging

hieronymus — hieronymus-omega.vercel.app

GPU-accelerated medical image analysis: segmentation, point-spread-function deconvolution, registration, and morphometry, with metric-preserving reconstruction and uncertainty quantification. Implemented in WebGL/GLSL for real-time pipelines. *Relevance*: direct experience with the image-analysis primitives underlying radiological AI — segmentation, registration, quantitative morphometry.

BRUT (rPPG) — remote-photoplethysmography.vercel.app

Camera-based physiological inference: recovers melanin, haemoglobin, and vasodilation from ordinary RGB video by inverting a layered optical skin model. *Relevance*: multimodal signal fusion — extracting several physically distinct physiological channels from one imaging stream, with an explicit forward model rather than a black box.

Laboratory & multi-omics data

lavoisier — lavoisier.vercel.app/sandbox

High-throughput mass-spectrometry pipeline: platform-invariant feature extraction, 98.9% accuracy against NIST reference libraries, distributed processing (Ray), memory-mapped I/O for large mzML datasets. Deterministic and benchmarked — reproducible by a third party from a stated seed. *Relevance*: the reproducible, reference-validated laboratory-data pipeline discipline the project asks for.

gospel — github.com/fullscreen-triangle/gospel

Multi-site multi-omics integration: host genotype, transcriptomics, and metagenomics as a joint Bayesian graph; validated against 1000 Genomes, gnomAD, and UK Biobank. *Relevance*: hands-on experience with what data actually flows across multiple clinical sites, where pseudonymisation fails, and the constraints multi-site analysis places on data governance.

ML engineering, agentic systems, and reproducibility

kwasa-kwasa / Turbulance — github.com/fullscreen-triangle/kwasa-kwasa

A declarative orchestration language for multi-agent, multi-substrate delegation (LLM oracles, tools, Python) with confidence/argumentation semantics and grounding of generated values. Rust core + TypeScript, with a VSCode language server.

four-sided-triangle — github.com/fullscreen-triangle/four-sided-triangle

An 8-stage metacognitive multi-agent RAG / reasoning pipeline with resource-aware orchestration; Rust core, FastAPI + Ray + Docker.

purpose — github.com/fullscreen-triangle/purpose

A Rust CLI for agent-queryable context retrieval that reduces LLM token cost; trait-based architecture in which hand-coded and LoRA-trained resolvers swap transparently.

Foundation-model adaptation & training methodology. LoRA fine-tuning and knowledge distillation; research on curriculum design (inverse curricula) for sample-efficient training. PyTorch throughout, at HPC/SLURM scale.

Reproducibility discipline (across all projects). Public source; validation suites that are deterministic and seeded, so results reproduce exactly; every quantitative claim checked against an explicit external reference rather than asserted. Version control, documentation, and containerised deployment as standard.